📌 Topic 2.9 — Taints & Tolerations

> **Module 2 — Workloads & Scheduling File:** `17-taints-and-tolerations.md` **CKA Weight:** ~15% (Workloads/Scheduling) — Taints appear in node management CKA tasks

---

🎯 What You'll Learn

- What taints and tolerations are and the problem they solve
- The 3 taint effects — NoSchedule, PreferNoSchedule, NoExecute
- How to taint and untaint nodes
- How to add tolerations to pods
- Built-in system taints and what they mean
- Taints vs Affinity — how they work together
- Real-world use cases (dedicated nodes, GPU, spot instances)
- Interview & scenario-based questions with answers

🧠 Core Concept Explained

The Allergy Analogy 🤧

> Think of taints and tolerations like **food allergies at a restaurant**:
>
> - **Taint on a node** = The restaurant puts up a sign: "⚠️ This kitchen contains NUTS"
> - **Toleration on a pod** = A customer who says: "I'm fine with nuts — I can eat here"
> - **Pod without toleration** = A customer with nut allergy: "I can't eat here — I'll go to another restaurant"
>
> Key insight: Taints **REPEL** pods from nodes. Tolerations **ALLOW** pods to be scheduled on tainted nodes.
>
> **Taints vs Affinity:**
>
> - **Affinity** = POD chooses which nodes it WANTS (attraction)
> - **Taints** = NODE repels pods it DOESN'T WANT (repulsion)
> - Together: full two-way scheduling control

🔬 What Are Taints & Tolerations?

```
TAINT (on NODE):
"This node repels pods that don't tolerate me"

Format: key=value:effect
Example: dedicated=gpu:NoSchedule

kubectl taint node node1 dedicated=gpu:NoSchedule
```

```
TOLERATION (on POD):
"This pod can tolerate the taint and run on that node"

Must MATCH the taint key, value, and effect
Added to pod spec under tolerations:
- key: "dedicated"
  value: "gpu"
  operator: "Equal"
  effect: "NoSchedule"
```

```
IMPORTANT: Tolerations ALLOW pods to be scheduled on tainted nodes
           but do NOT REQUIRE it. Use nodeAffinity to force placement.

Taint+Toleration = "you CAN go there"
Taint+Toleration+nodeAffinity = "you MUST go there"
```

⚡ The 3 Taint Effects

```
Effect 1: NoSchedule

→ New pods WITHOUT matching toleration: NOT scheduled here
→ Existing pods already running: UNAFFECTED (keep running)
→ Strictest scheduling restriction

Use: "Only specific pods should run on this node"
Example: GPU node, dedicated node, master node
```

```
┌─────────────────────────────────────────────────────────┐
│  Effect 2: PreferNoSchedule                             │
│                                                         │
│  → Scheduler TRIES to avoid placing pods here           │
│  → But WILL place here if no other node is available    │
│  → Soft version of NoSchedule                           │
│                                                         │
│  Use: "Prefer not to run here, but OK if necessary"     │
│  Example: Node undergoing maintenance soon              │
└─────────────────────────────────────────────────────────┘
```

```
┌─────────────────────────────────────────────────────────┐
│  Effect 3: NoExecute                                    │
│                                                         │
│  → New pods WITHOUT toleration: NOT scheduled here      │
│  → Existing pods WITHOUT toleration: EVICTED immediately │
│  → Existing pods WITH toleration: stay (optionally for N s) │
│                                                         │
│  Use: "Remove ALL pods from this node NOW"              │
│  Example: Node going into maintenance, node is unhealthy │
└─────────────────────────────────────────────────────────┘
```

```
EFFECT COMPARISON:
═══════════════════════════════════════════════════════
Effect           New pods (no tol)  Existing pods (no tol)
═══════════════════════════════════════════════════════
NoSchedule       NOT scheduled      KEEP RUNNING
PreferNoSchedule AVOID (soft)       KEEP RUNNING
NoExecute        NOT scheduled      EVICTED ❌
═══════════════════════════════════════════════════════
```

💻 Taint Commands

bash

```
# ── ADD TAINT ──────────────────────────────────────────────
# Format: kubectl taint node <node-name> <key>=<value>:<effect>
kubectl taint node node1 dedicated=gpu:NoSchedule
kubectl taint node node2 env=production:NoSchedule
kubectl taint node node3 maintenance=true:NoExecute

# Taint without value (key only)
kubectl taint node node1 special:NoSchedule

# Multiple taints on one node
kubectl taint node node1 dedicated=gpu:NoSchedule
kubectl taint node node1 accelerator=nvidia:PreferNoSchedule

# ── VIEW TAINTS ────────────────────────────────────────────
kubectl describe node node1 | grep Taint
# Taints: dedicated=gpu:NoSchedule

kubectl get nodes -o custom-columns=\
  NAME:.metadata.name,TAINTS:.spec.taints

# ── REMOVE TAINT ───────────────────────────────────────────
# Format: kubectl taint node <node> <key>=<value>:<effect>-
#                                                          ↑ trailing dash removes
kubectl taint node node1 dedicated=gpu:NoSchedule-

# Remove by key only (removes all effects with that key)
kubectl taint node node1 dedicated-

# Remove all taints (edit node directly)
kubectl edit node node1
# Delete the taints section from spec
```

📄 Toleration YAML — Full Reference

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: gpu-workload
spec:
  tolerations:

    # ── MATCH EXACT TAINT ──────────────────────────────
    - key: "dedicated"
      operator: "Equal"           # Equal | Exists
      value: "gpu"
      effect: "NoSchedule"
    # Matches taint: dedicated=gpu:NoSchedule ✅

    # ── MATCH BY KEY ONLY (any value) ──────────────────
    - key: "dedicated"
      operator: "Exists"          # matches any value for this key
      effect: "NoSchedule"
    # Matches: dedicated=gpu:NoSchedule ✅
    # Matches: dedicated=cpu:NoSchedule ✅
    # Matches: dedicated=anything:NoSchedule ✅

    # ── MATCH ALL TAINTS (wildcard) ────────────────────
    - operator: "Exists"          # no key = match ALL taints!
    # Matches EVERY taint on the node
    # Use carefully — allows pod anywhere

    # ── NoExecute WITH TIME LIMIT ──────────────────────
    - key: "node.kubernetes.io/not-ready"
      operator: "Exists"
      effect: "NoExecute"
      tolerationSeconds: 300      # stay for 5 min, then evict
    # Pod stays on NotReady node for 5 min before being evicted
    # Without this: evicted immediately on NoExecute

  containers:
    - name: gpu-trainer
      image: tensorflow:2.9-gpu
```

🔄 Toleration Matching Rules

```
MATCHING LOGIC:

Taint on node:    dedicated=gpu:NoSchedule

Pod toleration:   key: dedicated, value: gpu, effect: NoSchedule
                  operator: Equal
Result: MATCH ✅ (pod can be scheduled here)

Pod toleration:   key: dedicated, operator: Exists, effect: NoSchedule
Result: MATCH ✅ (Exists ignores value)

Pod toleration:   key: dedicated, value: cpu, effect: NoSchedule
                  operator: Equal
Result: NO MATCH ❌ (value mismatch: gpu ≠ cpu)

Pod toleration:   key: dedicated, value: gpu, effect: PreferNoSchedule
                  operator: Equal
Result: NO MATCH ❌ (effect mismatch: NoSchedule ≠ PreferNoSchedule)

Pod toleration:   key: other-key, value: gpu, effect: NoSchedule
Result: NO MATCH ❌ (key mismatch)

Pod toleration:   operator: Exists (no key, no effect)
Result: MATCH ✅ ALL taints (wildcard — matches everything)

RULE: Pod must have a matching toleration for EVERY taint on the node
      to be scheduled there. One unmatched taint blocks scheduling.
```

🏛️ Built-in System Taints

Kubernetes automatically applies these taints to nodes in specific situations:

```
┌─────────────────────────────────────────────────────────────┐
│  AUTOMATIC TAINTS (added by node lifecycle controller)      │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  node.kubernetes.io/not-ready:NoExecute                     │
│  → Node is NotReady (kubelet not reporting)                 │
│  → Pods evicted after tolerationSeconds (default 300s)      │
│                                                             │
│  node.kubernetes.io/unreachable:NoExecute                   │
│  → Node is unreachable (network partition)                  │
│  → Pods evicted after tolerationSeconds (default 300s)      │
│                                                             │
│  node.kubernetes.io/out-of-disk:NoSchedule                  │
│  → Node has no disk space                                   │
│                                                             │
│  node.kubernetes.io/memory-pressure:NoSchedule              │
│  → Node is under memory pressure                            │
│                                                             │
│  node.kubernetes.io/disk-pressure:NoSchedule                │
│  → Node has low disk space                                  │
│                                                             │
│  node.kubernetes.io/pid-pressure:NoSchedule                 │
│  → Node has too many processes                              │
│                                                             │
│  node.kubernetes.io/unschedulable:NoSchedule                │
│  → Node is cordoned (kubectl cordon)                        │
│                                                             │
│  node-role.kubernetes.io/control-plane:NoSchedule           │
│  → Control plane node (added by kubeadm)                    │
│  → Prevents regular pods from running on master             │
│                                                             │
└─────────────────────────────────────────────────────────────┘

Default tolerations on every pod (added automatically by K8s):
  node.kubernetes.io/not-ready:NoExecute    tolerationSeconds: 300
  node.kubernetes.io/unreachable:NoExecute  tolerationSeconds: 300

→ This is why pods stay on nodes for 5 minutes after node failure
  before being rescheduled elsewhere!
```

🌍 Real-World Use Cases

Use Case 1: Dedicated GPU Nodes

```bash
# Problem: GPU nodes are expensive, only ML jobs should use them
# Regular pods accidentally get scheduled there too

# Solution:
# 1. Taint all GPU nodes
kubectl taint node gpu-node-1 dedicated=gpu:NoSchedule
kubectl taint node gpu-node-2 dedicated=gpu:NoSchedule

# 2. Add toleration ONLY to ML workloads
# 3. Add nodeAffinity to FORCE ML workloads to GPU nodes

# Result:
# Regular pods → cannot land on GPU nodes (taint repels them) ✅
# ML pods → toleration allows them, affinity forces them there ✅
```

```yaml
# ML training pod
spec:
  tolerations:
    - key: dedicated
      value: gpu
      operator: Equal
      effect: NoSchedule
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: dedicated
                operator: In
                values: ["gpu"]
```

Use Case 2: Control Plane Isolation

```bash
```

```
# kubeadm adds this taint automatically:
# node-role.kubernetes.io/control-plane:NoSchedule

# Regular pods cannot run on control plane ✅
# System pods (CoreDNS, kube-proxy) have tolerations ✅

# To allow a specific pod on control plane:
tolerations:
  - key: node-role.kubernetes.io/control-plane
    operator: Exists
    effect: NoSchedule
```

Use Case 3: Node Maintenance

```bash
# Prepare node for maintenance without downtime:

# Step 1: Cordon (adds unschedulable taint)
kubectl cordon node1
# Adds: node.kubernetes.io/unschedulable:NoSchedule
# → No new pods scheduled here

# Step 2: Drain (evicts all pods gracefully)
kubectl drain node1 \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --grace-period=60
# Adds: NoExecute taint → pods gracefully terminated

# Step 3: Do maintenance on node1

# Step 4: Uncordon (removes taint)
kubectl uncordon node1
# Removes unschedulable taint → pods can schedule again
```

Use Case 4: Spot/Preemptible Instances

```bash
```

```
# Spot instances can be terminated anytime
# Taint them so only fault-tolerant workloads run there

kubectl taint node spot-node-1 spot=true:NoSchedule
kubectl taint node spot-node-2 spot=true:NoSchedule

# Only batch jobs (tolerant of interruption) get toleration:
tolerations:
  - key: spot
    value: "true"
    operator: Equal
    effect: NoSchedule
```

Use Case 5: Node Problem Detection

```yaml
# Using tolerationSeconds for graceful degradation
tolerations:
  - key: node.kubernetes.io/not-ready
    operator: Exists
    effect: NoExecute
    tolerationSeconds: 60    # wait 1 min before evicting
                             # gives node time to recover
                             # default is 300s (5 min)

  - key: node.kubernetes.io/unreachable
    operator: Exists
    effect: NoExecute
    tolerationSeconds: 60
```

🔀 Taints + Affinity Together

```
JUST TAINT (without affinity):
  Node1: taint=gpu:NoSchedule
  GPU pod: toleration for gpu taint

  Result: GPU pod CAN go to gpu-node ✅
          GPU pod CAN ALSO go to non-gpu nodes ✅
          → pod might not land on GPU node!
```

```
TAINT + AFFINITY (complete control):
  Node1: taint=gpu:NoSchedule  +  label: accelerator=gpu
  GPU pod: toleration          +  nodeAffinity required: accelerator=gpu

  Result: GPU pod CAN go to gpu-node (toleration) ✅
          GPU pod MUST go to gpu-node (affinity) ✅
          Regular pods CANNOT go to gpu-node (taint) ✅
          → Perfect bidirectional control!

VISUALIZATION:
  Taint/Toleration = NODE says "I don't want these pods"
  Affinity = POD says "I want to go to that node"
  Together = Complete scheduling control
```

⚠️ Common Mistakes & Gotchas

Mistake	Reality
Toleration alone guarantees placement	Toleration only ALLOWS — use nodeAffinity to REQUIRE
NoSchedule evicts existing pods	NoSchedule only affects NEW pods — use NoExecute to evict
Removing taint automatically reschedules	Removing taint doesn't move existing pods — they stay where they are
Forgetting `operator: Exists` for no-value taints	`operator: Equal` requires a `value` field
Using `operator: Exists` without key	Matches ALL taints — very permissive, use carefully
Tolerating `not-ready` without `tolerationSeconds`	Pod never gets evicted from dead node! Always set tolerationSeconds

---

🎓 CKA Exam Tips

```
✅ Taint format: key=value:effect  OR  key:effect (no value)
   kubectl taint node <n> key=value:NoSchedule
```

```
✅ Remove taint: add trailing dash
   kubectl taint node <n> key=value:NoSchedule-
   kubectl taint node <n> key-  (removes all effects for key)

✅ View taints:
   kubectl describe node <n> | grep -i taint

✅ Effect names (capitalized exactly):
   NoSchedule | PreferNoSchedule | NoExecute

✅ Control plane taint (common in CKA):
   node-role.kubernetes.io/control-plane:NoSchedule

✅ Toleration for control plane pods:
   - key: node-role.kubernetes.io/control-plane
     operator: Exists
     effect: NoSchedule

✅ kubectl drain adds NoExecute taint automatically
   kubectl cordon adds unschedulable taint

✅ If asked to "allow pod to run on tainted node":
   → Add matching toleration to pod spec
   → If also need to FORCE it there: add nodeAffinity too
```

❓ Interview & Scenario Questions

Q1: What is the difference between taints/tolerations and nodeAffinity?

Answer: Taints and tolerations work from the **node's perspective** — a tainted node repels pods, and tolerations on pods allow them to bypass that repulsion. NodeAffinity works from the **pod's perspective** — a pod declares which nodes it wants or prefers. They serve complementary purposes: taints protect nodes from unwanted pods (e.g., dedicated GPU node shouldn't have regular workloads), while affinity directs pods to specific nodes (e.g., ML workload should go to GPU nodes). For complete scheduling control, combine both: taint to repel unwanted pods + affinity to attract desired pods.

Q2: Explain the three taint effects and when to use each.

Answer:

- **NoSchedule**: New pods without matching toleration won't be scheduled on this node. Existing pods are unaffected. Use when setting up a dedicated node going forward without disturbing running workloads.
- **PreferNoSchedule**: Soft version — the scheduler avoids this node but will use it if no alternatives exist. Use when you prefer pods avoid a node but don't want to hard-block.
- **NoExecute**: New pods without toleration won't be scheduled AND existing pods without toleration are evicted. Use for node maintenance, marking unhealthy nodes, or immediate evacuation. Supports `tolerationSeconds` to allow graceful eviction after a delay.

Q3: Scenario — You want ONLY your data-processing pods to run on nodes labeled `workload=batch`. All other pods should be blocked. How?

Answer: Use both taint AND nodeAffinity:

```bash
# 1. Taint the batch nodes (repels all pods without toleration)
kubectl taint node batch-node-1 workload=batch:NoSchedule
kubectl taint node batch-node-2 workload=batch:NoSchedule
```

```yaml
# 2. Add to data-processing pod spec:
spec:
  tolerations:
    - key: workload
      value: batch
      operator: Equal
      effect: NoSchedule           # allows pod on batch nodes
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: workload
                operator: In
                values: ["batch"]  # forces pod to batch nodes
```

Result: Batch nodes only get data-processing pods (taint blocks others + affinity forces this pod there).

Q4: Why do pods stay running for 5 minutes on a NotReady node before being rescheduled?

Answer: Every pod automatically gets two default tolerations added by Kubernetes:

```yaml
- key: node.kubernetes.io/not-ready
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 300
- key: node.kubernetes.io/unreachable
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 300
```

When a node becomes NotReady, the node lifecycle controller adds a `not-ready:NoExecute` taint. Pods tolerate this taint for 300 seconds (5 minutes), giving the node time to recover. If the node recovers within 5 minutes, pods continue running without disruption. If not, after 300 seconds the pods are evicted and rescheduled on healthy nodes.

Q5: What happens when you `kubectl drain` a node?

Answer: `kubectl drain` performs two actions: (1) **Cordons** the node by marking it unschedulable (adds `unschedulable:NoSchedule` taint) so no new pods are scheduled. (2) **Evicts** all pods from the node gracefully — it respects PodDisruptionBudgets, waits for the grace period (`terminationGracePeriodSeconds`), and recreates them on other nodes via their controllers. DaemonSet pods are not evicted by default (use `--ignore-daemonsets`). Pods using `emptyDir` volumes lose their data — use `--delete-emptydir-data` to allow their eviction. Static pods cannot be evicted but can be deleted by removing their manifest files.

📚 Quick Summary / Cheatsheet

```
┌──────────────────────────────────────────────────────────────┐
│                TOPIC 2.9 — TAINTS & TOLERATIONS               │
├──────────────────────────────────────────────────────────────┤
│  TAINT = node repels pods  (push away)                        │
│  TOLERATION = pod can bypass taint  (allow in)                │
│  Format: key=value:effect  OR  key:effect                     │
├──────────────────────────────────────────────────────────────┤
```

```
│  3 EFFECTS:                                                  │
│  NoSchedule       → new pods blocked, existing kept          │
│  PreferNoSchedule → soft block (scheduler avoids)            │
│  NoExecute        → new pods blocked + existing EVICTED      │
│                     (supports tolerationSeconds for delay)   │
├──────────────────────────────────────────────────────────────┤
│  KEY COMMANDS:                                               │
│  kubectl taint node <n> key=val:effect                       │
│  kubectl taint node <n> key=val:effect-  (remove)            │
│  kubectl taint node <n> key-             (remove all)        │
│  kubectl describe node <n> | grep Taint                      │
├──────────────────────────────────────────────────────────────┤
│  TOLERATION OPERATORS:                                       │
│  Equal  → must match key + value + effect                    │
│  Exists → match key only (any value); no key = match all     │
├──────────────────────────────────────────────────────────────┤
│  SYSTEM TAINTS:                                              │
│  control-plane:NoSchedule → kubeadm master node              │
│  not-ready:NoExecute      → unhealthy node (auto)            │
│  unreachable:NoExecute    → network partition (auto)         │
│  unschedulable:NoSchedule → kubectl cordon (auto)            │
├──────────────────────────────────────────────────────────────┤
│  BEST PRACTICE:                                              │
│  Taint + Toleration = pod CAN go to node                     │
│  Taint + Toleration + nodeAffinity = pod MUST go to node     │
└──────────────────────────────────────────────────────────────┘
```

> **Next Topic →** `18-scheduler-deep-dive.md` *How the scheduler works internally, manual scheduling, custom schedulers, priority classes*